

Practical SQL How-To Guide

A beginner-friendly reference for querying, cleaning, validating, and troubleshooting relational data

Practical Tools & Guides | Built for real-world reference, learning, and repeatable work.

How to use this guide

Examples use MariaDB-style SQL unless noted. The goal is not to memorize every command. It is to know the common patterns, the order they go in, and where to look when your brain is not braining.

1. The basic mental model

A relational database stores information in tables. Each table should represent one type of thing or event. Relationships connect those tables so you can answer questions without repeating the same data everywhere.

Term	Plain-English meaning
Table	A structured set of rows and columns.
Row	One record, such as one employee or one invoice.
Column	One attribute, such as employee_id or invoice_total.
Primary key	The value that uniquely identifies a row.
Foreign key	A value that points to a primary key in another table.
NULL	No value is present. It is different from 0 or an empty string.
Join	A way to combine related rows from multiple tables.

2. The SELECT order you will use constantly

```
SELECT column1, column2
FROM table_name
JOIN other_table
  ON table_name.key = other_table.key
WHERE condition
GROUP BY column1, column2
HAVING aggregate_condition
ORDER BY column1;
```

Easy order to remember

SELECT what you want, FROM where it lives, JOIN what it relates to, WHERE which rows, GROUP BY how to summarize, HAVING which groups, ORDER BY how to display.

3. Filtering rows

```
SELECT *  
FROM employees  
WHERE employee_status = 'Active';
```

```
SELECT *  
FROM expenses  
WHERE amount > 1000  
AND expense_date >= '2026-01-01';
```

Useful operators: =, != or <>, >, <, >=, <=, BETWEEN, IN, LIKE, IS NULL, IS NOT NULL.

4. Sorting and limiting results

```
SELECT employee_id, employee_name, hire_date  
FROM employees  
ORDER BY hire_date DESC  
LIMIT 10;
```

5. Aggregation

```
SELECT department,  
    COUNT(*) AS employee_count,  
    AVG(annual_salary_hourly_rate) AS average_rate  
FROM employees  
GROUP BY department  
ORDER BY employee_count DESC;
```

6. WHERE vs HAVING

WHERE filters individual rows before grouping. HAVING filters the grouped result after aggregation.

```
SELECT department, COUNT(*) AS employee_count  
FROM employees  
WHERE employee_status = 'Active'  
GROUP BY department  
HAVING COUNT(*) >= 5;
```

7. JOINS

Join	Use it when...
INNER JOIN	You only want rows that exist in both tables.
LEFT JOIN	You want every row from the left table, even when no match exists.
SELF JOIN	A table relates to itself, such as employee -> supervisor.

```
SELECT i.invoice_id,  
       c.customer_name,  
       i.invoice_total  
FROM invoices i  
JOIN customers c  
  ON i.customer_id = c.customer_id;
```

8. NULLs and blank strings

NULL is not equal to anything, including another NULL. Use IS NULL and IS NOT NULL.

```
SELECT *  
FROM customer_addons  
WHERE end_date IS NULL;
```

To turn blank strings into NULL during cleanup:

```
NULLIF(TRIM(column_name), '')
```

9. CASE for normalization and business rules

```
CASE  
  WHEN department IN ('Field Svc', 'Field Services')  
    THEN 'Field Services'  
  WHEN department IN ('Software/Data', 'Software & Data')  
    THEN 'Software & Data'  
  ELSE TRIM(department)  
END
```

10. Finding duplicates

```
SELECT employee_id,  
       COUNT(*) AS row_count  
FROM stg_hris_employee_roster  
GROUP BY employee_id  
HAVING COUNT(*) > 1;
```

Do not delete duplicate-looking records until you compare the full fields. Two rows can share an ID/date/action and still represent different events.

11. Window functions for controlled deduplication

```
SELECT *,
    ROW_NUMBER() OVER (
        PARTITION BY employee_id, effective_date, action
        ORDER BY staging_row_id
    ) AS duplicate_rank
FROM staging_table;
```

Then keep `duplicate_rank = 1` only after you have proved the other rows are true duplicates.

12. Data type conversion

```
CAST(value AS DECIMAL(12,2))
CAST(value AS UNSIGNED)
STR_TO_DATE(date_text, '%Y-%m-%d')
```

13. Excel serial dates

```
DATE_ADD(
    '1899-12-30',
    INTERVAL CAST(@raw_date AS UNSIGNED) DAY
)
```

Use this only when the source really contains Excel serial values such as 45958. Do not use it on normal text dates like 2026-09-15.

14. Floating-point cleanup

```
ROUND(
    CAST(@raw_hours AS DECIMAL(20,10)),
    2
)
```

Useful when a source exports 158.9 as 158.89999999999998.

15. Safe UPDATE habit

1. Write the WHERE condition as a SELECT first.
2. Confirm the rows are exactly the rows you intend to change.
3. Then run the UPDATE using the same WHERE condition.
4. Check affected rows and warnings.

```
SELECT *
FROM invoice_lines
WHERE TRIM(project_id) = '';
```

```
UPDATE invoice_lines
SET project_id = NULL
WHERE TRIM(project_id) = '';
```

16. Foreign keys

```
ALTER TABLE expenses
ADD CONSTRAINT fk_expenses_employee
FOREIGN KEY (employee_id)
REFERENCES employees(employee_id);
```

Validate unmatched values before adding a foreign key. A foreign key protects future integrity, but it does not replace business-rule validation.

17. The 'zero is good' validation pattern

Instead of only asking whether a report number looks right, ask how many records violate the rule. For many validation checks, the desired answer is zero.

```
SELECT COUNT(*) AS invalid_rows
FROM invoices
WHERE ROUND(invoice_total - amount_paid, 2)
    <> ROUND(balance_due, 2);
```

18. Common validation patterns

```
-- Orphans
SELECT COUNT(*)
FROM child c
LEFT JOIN parent p ON c.parent_id = p.parent_id
WHERE p.parent_id IS NULL;
```

```
-- Invalid date order
SELECT COUNT(*)
FROM projects
WHERE end_date < start_date;
```

```
-- Values outside an allowed range
SELECT COUNT(*)
FROM product_usage_monthly
WHERE active_user_pct < 0
    OR active_user_pct > 100;
```

```
-- Cross-table mismatch
```

```
SELECT COUNT(*)
FROM time_entries t
JOIN projects p ON t.project_id = p.project_id
WHERE t.customer_id <> p.customer_id;
```

19. Import pattern with LOAD DATA

```
LOAD DATA LOCAL INFILE
'C:/Path/File.csv'
INTO TABLE target_table
CHARACTER SET utf8mb4
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(
    id,
    @nullable_value,
    amount
)
SET
    nullable_value = NULLIF(TRIM(@nullable_value), "");
```

Always check three numbers

Records = expected count. Skipped = 0 unless intentionally skipped. Warnings = 0 unless you deliberately investigated and accepted them.

20. Debugging checklist

- Records: 0? Check line endings, header rows, file path, and whether the table is already populated.
- Data truncated? Inspect the raw value and target data type.
- File not found? Verify the exact local Windows filename and folder. A filename shown in a chat upload may not match your PC.
- Foreign key failure? Find unmatched IDs before changing constraints.
- Unexpected count? Check the exact status labels and business terminology before assuming the data is wrong.
- Old warnings still visible in your client? Confirm which statement produced them.

21. SQL dialects: the transferable part

Core SQL concepts transfer well across MariaDB/MySQL, PostgreSQL, SQL Server, Oracle, and SQLite. Expect differences around dates, generated IDs, row limiting, string functions, procedures, imports, and vendor-specific features.

Concept	MariaDB/MySQL	SQL Server	PostgreSQL / Oracle notes
Limit rows	LIMIT 10	TOP 10	PostgreSQL: LIMIT 10; Oracle: FETCH FIRST 10 ROWS ONLY
Auto number	AUTO_INCREMENT	IDENTITY(1,1)	PostgreSQL: IDENTITY/SERIAL; Oracle: IDENTITY or sequence
Add 30 days	DATE_ADD(d, INTERVAL 30 DAY)	DATEADD(DAY,30,d)	Different date arithmetic syntax
Null fallback	IFNULL(x,0)	ISNULL(x,0)	COALESCE(x,0) is broadly portable